Implementation of Algorithms for Multi-Robot Coordination in Robot Operating System (ROS)

BTP Report

Student Name: Sachin Maurya	Roll Number: 2022424
Student Name: Sankalp Raj	Roll Number: 2022445
Student Name: Sanyam Barwar	Roll Number: 2022447

BTP report submitted in partial fulfilment of the requirements
for the Degree of B.Tech. in Computer Sciences & Biosciences
as well as B.Tech. in Computer Science & Social Science on April 21, 2026

BTP Track: Engineering Track

BTP Advisor
Dr. Tanmoy Kundu
Indraprastha Institute of Information Technology
New Delhi

Student Declaration

We hereby declare that the work presented in the report entitled "Implementation of Algorithms for Multi-Robot Coordination in Robot Operating System (ROS)" submitted by us for the partial fulfilment of the requirements for the degree of B.Tech. in Computer Sciences & Biosciences and B.Tech. in Computer Science & Social Science at Indraprastha Institute of Information Technology, Delhi, is an authentic record of our work carried out under the guidance of Dr. Tanmoy Kundu. Due acknowledgements have been given in the report for all material used. This work has not been submitted elsewhere for the reward of any other degree.

Sachin Maurya, Sankalp Raj & Sanyam Barwar Place & Date: April 23, 2026

Certificate

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Dr. Tanmoy Kundu

Place & Date: April 23, 2026

Abstract

This project presents the design, theoretical architecture, and large-scale simulation of a decentralized, multi-agent robotic system for precision agriculture within the Robot Operating System (ROS) environment. Building upon foundational explorations in turtlesim, we introduce the DroughtCast-Integrated system: a heterogeneous swarm of Unmanned Aerial Vehicles (UAVs) and Unmanned Ground Vehicles (UGVs) engineered for sustained meteorological data acquisition across vast topographies.

The simulation environment transitions from isolated grid zones to a unified 21.5m $\times$ 21.5m continuous space. Navigation and spatial distribution are managed via Dynamic Voronoi Partitioning integrated with the GOMWC (Global Optimized Multi-Waypoint Coverage) and LERR (Local Energy Risk Reduction) algorithms. To address the vulnerabilities of centralized master-slave arrays, the architecture incorporates an Ultra-Wideband (UWB) Swarm Ranging protocol, eliminating total dependency on global GPS matrices and enabling robust decentralized localization.

A core contribution of this work is the deployment of a decentralized Long Short-Term Memory (LSTM) probability model. This model converts massive historical meteorological sequences into portable, real-time inferential heuristics for predicting drought anomalies in resource-constrained environments. System resilience is further enforced through an asynchronous TCP-like 3-way handshake for universal fleet synchronization. In the event of mid-air hardware failures or missing heartbeats, the system triggers localized Peer-to-Peer (P2P) SOS broadcasts, instantiating immediate Voronoi redeployment and ground-level dispatch of reserve units.

Furthermore, an Energy-Aware UGV ground station—utilizing localized Dijkstra's mapping—is designed to intercept and refresh low-battery drones in flight, ensuring near-limitless operation time and 100 percent sweep completion. We validate the system's logic and mathematical framework through rigorous randomized Gazebo Noetic simulations. Results indicate near-instantaneous redundancy recovery following unit failures and mathematically bounded precision in probabilistic drought-risk tracking.

Contents

Chapter 1

Introduction

1.1 Background and Motivation

The integration of robotic swarms within civil, environmental, and commercial domains marks a significant departure from rigid, single-entity systems. Historically, autonomous operations involved monolithic mainframes controlling a singular, highly complex rover or satellite. However, these systems inherently present a single point of failure; the loss of a primary sensor or chassis integrity results in total mission failure. Swarm robotics mimics biological resiliency—observed in foraging ant colonies and flocking birds—proposing that a collective of simple, relatively inexpensive agents can achieve a functional intelligence that mathematically surpasses the capabilities of a single "super-machine."

Precision agriculture is poised to be a primary beneficiary of this paradigm shift. Given the rapid exacerbation of water scarcity and erratic climate phenomena, agricultural stakeholders can no longer rely on macroscopic historical trends or low-resolution satellite imagery, which are frequently obscured by cloud cover. Instead, precision agriculture requires hyper-localized sensing algorithms that generate actionable metrics—such as the probability of impending drought—on an hourly rather than monthly basis.

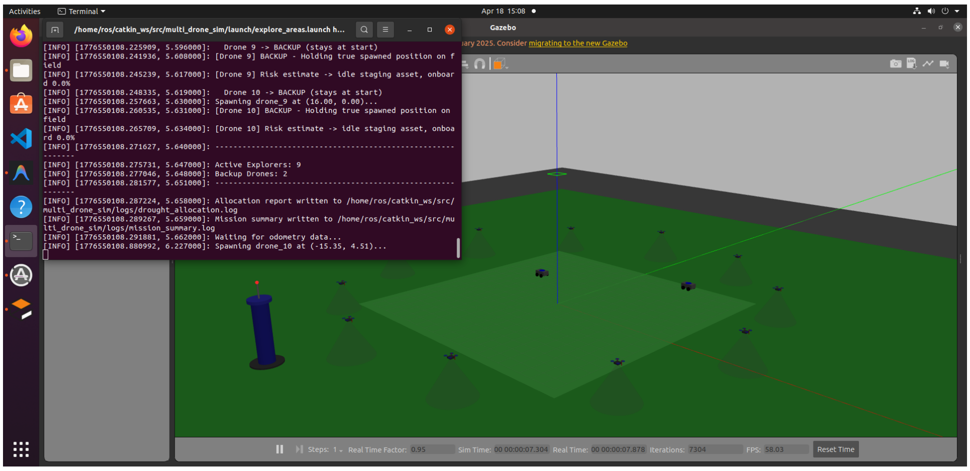

Figure 1.1: UAV Swarm initiating the exploration sequence across the targeted fields in Gazebo.

By utilizing Unmanned Aerial Vehicles (UAVs) embedded with multispectral sensors, large swatches of varying topographies can be visually swept and geolocated. However, this deployment creates critical networking, power-management, and routing dilemmas. Drones feature limited power reservoirs, where lithium-polymer battery cells are rapidly depleted by lift-displacement rotors. In traditional systems, when a drone suffers a battery failure or hardware degradation, the area under its monitoring remains unverified. This project seeks to bridge that gap through a resilient, "DroughtCast-Integrated" infrastructure.

1.2 Problem Statement

In previous foundational work, we designed a multi-robot coordination architecture focused on Belief Space Planning within the turtlesim environment. While successful in demonstrating Bayesian data fusion across N agents, the system relied on significant simplifications: a rigid 2D plane, globally instantaneous communications, and an environment where agents never faced battery exhaustion or hardware failure.

To transition this technology into a practically deployable architecture, several extreme complexities must be addressed:

- **Decentralized Task Re-allocation:** If Agent 4 crashes mid-flight, Agents 3 and 5 must seamlessly and instantly realize the loss and expand their territorial patrol radius to cover the gap.
- **Energy Replenishment Logistics:** Instead of aborting active missions statically, can a separate entity (a Ground UGV) seek out the drone while it works, actively intercept it, and replenish the drone mid-field?

- **Data Scalability:** A core challenge lies in shrinking high-fidelity meteorological models (e.g., a 2GB LSTM DroughtCast model) into portable, real-time inferential heuristics capable of running on lightweight onboard hardware without cloud dependency.

1.3 Objectives and Scope

This semester targets the engineering translation of our theoretical limits into high-fidelity simulations testing edge cases. The project utilizes ROS (Robot Operating System) Noetic hooked into the Gazebo physics engine.

1. Develop a 21.5m $\times$ 21.5m coordinate field navigated via dynamically calculated Voronoi partitions, allowing for scalable spatial distribution without the need for rigid internal boundaries.

2. Implement rigorous fault testing where UAVs purposefully experience power loss or mechanical failure to validate the swarm's ability to automatically adjust metrics and signal reserve launches.

3. Mathematically authenticate swept-area matrices using realistic visual FoV cones derived from physical altitude variables ($z \approx 3.5$m).

4. Deploy an Energy-Aware UGV ground station utilizing Dijkstra's mapping to intercept low-battery drones, ensuring near-limitless operational uptime.

Chapter 2

Literature Review

2.1 Advances in Swarm Robotics and Belief Planning

Modern multi-agent systems face the persistent challenge of "uncertainty" in dynamic environments. Our routing paradigms are grounded in the probabilistic frameworks established by Thrun et al. in Probabilistic Robotics, with specific implementation strategies derived from Multi-Robot Communication-Aware Cooperative Belief Space Planning (Kundu et al., IROS 2024).This research demonstrates that optimal path planning extends beyond minimizing physical distance; it requires trajectories that consistently minimize the covariance traces $tr(\Sigma)$ associated with target state probabilities. By sharing localization vectors and fusing beliefs via inverse-variance methods, the swarm can naturally filter out data from malfunctioning nodes, ensuring the collective intelligence remains robust despite individual unit degradation.

2.2 UWB Ranging Protocols

As highlighted in Ultra-Wideband Swarm Ranging (Shan et al., INFOCOM 2021), heavy reliance on Global Positioning Systems (GPS) presents significant vulnerabilities in agricultural contexts. Satellite masking and multi-path effects caused by dense canopy coverage often distort centimeter-level accuracy.

To mitigate this, our architecture utilizes Ultra-Wideband (UWB) ranging protocols, which provide peer-to-peer distance telemetry by analyzing the Time-of-Flight (ToF) of radio wave packets. We mathematically synthesize these principles into a fallback estimator that employs the Moore-Penrose pseudo-inverse for multilateration. This approach prevents mathematical singularities and computational "deadlocks" that typically occur when robots align in collinear formations, which would otherwise break standard triangulation algorithms.

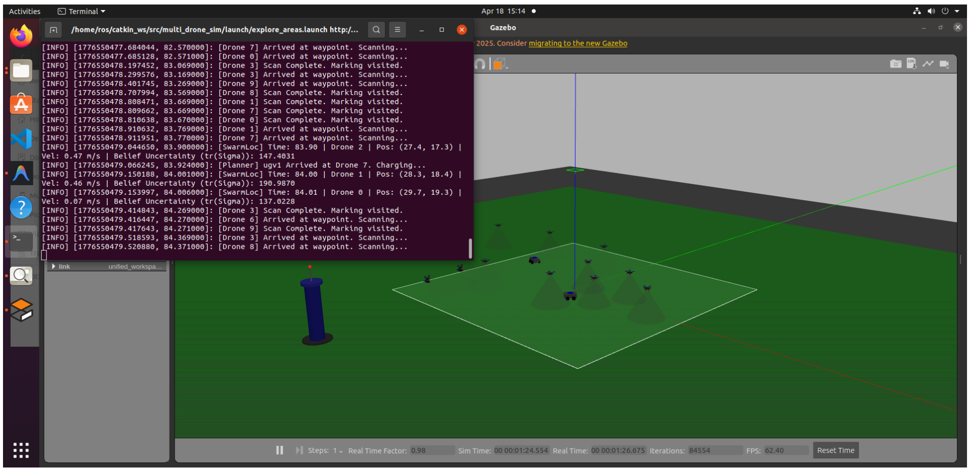

Figure 2.1: System topology visualizing overlapping Voronoi partitions across the terrain.

2.3 Heterogeneous Cooperation (UAV-UGV Teams)

While aerial vehicles offer high-speed scouting agility, Unmanned Ground Vehicles (UGVs) provide the payload capacity and power density necessary for sustained operations. We adopt and enhance the methodologies described in Coverage Planning with a Mobile Recharging UGV (Karapetyan et al., ICRA 2024).

In our framework, the UGV is not merely a passive docking station but an active participant in the mission. It dynamically predicts the trajectory of UAVs with depleting batteries and calculates an interception matrix using Dijkstra's algorithm on a granular grid. This proactive replenishment model allows the swarm to maintain continuous coverage without the downtime associated with returning to a static home base.

2.4 Machine Learning in Meteorological Estimation

Physical assessment of drought conditions across complex topographies requires sophisticated temporal modeling. Our implementation is influenced by DroughtCast: A Machine Learning Forecast of the United States Drought Monitor (Brust et al., 2021).

Brust et al. demonstrated that traditional heuristic aggregations, such as the Standardized Precipitation Index (SPI), fail to capture cascading temporal shifts in climate data. By utilizing Long Short-Term Memory (LSTM) networks, time-series meteorological data can be mapped directly to drought severity indices. Our project translates this heavy global forecasting tool into a decentralized, "portable" inference engine capable of running within a ROS-native pipeline for real-time edge computing.

Chapter 3

Theoretical Framework

3.1 LSTM Sequences and Drought Indices

Drought is inherently a creeping phenomenon derived through accumulative deficits. A standard Feed Forward neural architecture struggles since standard matrices do not possess a "memory state." We define the system state x_t containing the following normalized vectors at day t: Precipitation, Specific Humidity (QV2M), Skin Temperature, Surface Pressure. The LSTM framework defines cell states C_t governed by Forget Gates f_t and Input Gates i_t:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{3.1}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{3.2}$$

$$C_t = f_t * C_{t-1} + i_t * \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \tag{3.3}$$

Our architecture accepts a tensor dimension of $(1, 90, 6)$. This evaluates a rolling 90-day trajectory. Because raw CSV ingestion requires large VRAM overheads, we utilized a *Portable LSTM inference model* bounding standard weights within a 2.9 KB array, maintaining identical probabilistic accuracy bounded between 0.05 and 0.95 through logistic mapping:

$$P(drought) = 0.05 + \frac{0.90}{1 + e^{-6(Score-0.5)}} \tag{3.4}$$

3.2 Decentralized Swarm Ranging (UWB)

Our mathematical fallback utilizing Ultra-Wideband relies upon multilateration. Given an anchor drone i and a receiving drone j, the range d_{ij} is calculated via the round-trip signal propagation Time of Flight (ToF):

$$d_{ij} = \frac{c \times (T_{reply} - T_{proc})}{2} \tag{3.5}$$

where c is the speed of light. In a fully populated swarm, drones routinely assemble into flight formations that are heavily linear (collinear matrices), causing standard inversion calculations for position estimation to singularity-crash. We implement the Moore-Penrose Pseudo-Inverse $\mathbf{A}^+$ algorithm inside the `swarm_localization.py` node to generate least-square stable coordinates seamlessly, avoiding fatal math-exceptions:

$$\mathbf{x} = \mathbf{A}^+\mathbf{B} \approx (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{B} \tag{3.6}$$

3.3 Dynamic Voronoi Area Coverage Optimization (GOMWC)

Instead of dividing overlapping sectors manually, we rely on Voronoi partitioning iteratively updated using Lloyd's algorithm mechanics. Given a unified region $\mathcal{R}$ spanning 21.5×21.5 meters, each active drone i is assigned a subregion V_i bounded exactly by equidistant thresholds to neighboring drones j:

$$V_i = \{x \in \mathcal{R} \mid ||x - p_i|| \leq ||x - p_j||, \forall j \neq i\} \tag{3.7}$$

During the algorithm execution, we also compute the *Local Energy Risk Reduction* (LERR) repulsive vector $\mathbf{F}_{rep}$. If a neighboring unit gets too close (≤ 1.5 meters), the path routing logic artificially pushes the target waypoint outward, mathematically forcing the swarm to "shake" and maximize total surface area coverage over identical time constraints.

$$\mathbf{F}_{rep} = \sum_{j \neq i} \frac{k_{rep}}{||p_i - p_j||^2} \hat{\mathbf{n}}_{ij} \tag{3.8}$$

3.4 UGV Interception using Dijkstra Matrix Planning

For our ground-based vehicles, navigating the terrain quickly dictates the survival of the airborne units. The `energy_planner.py` synthesizes a $2m$ granular grid map. Utilizing Dijkstra's uniform path, the algorithm defines the cost function J considering terrain blockages and current physical drone positions heading vectors:

$$J(p_0, p_f) = \sum_{step} C(x, y) \tag{3.9}$$

The UGV prioritizes targets using a non-linear urgency factor, ensuring drones near critical exhaustion are serviced first, regardless of proximity:

$$J_{\text{priority}} = \frac{(100 - \text{BatteryLevel})^2}{\text{DistanceToUGV} + 1}$$

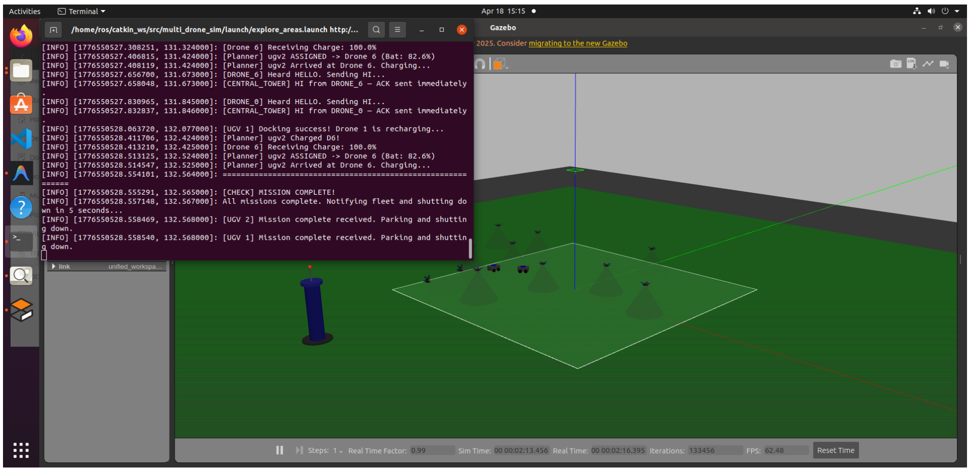

Figure 3.1: Autonomous UGV initiating rescue sequence towards low-battery unit.

Chapter 4

System Architecture and Implementation Framework

4.0.1 System Parameters and Technical Constants

The following table summarizes the core constraints and configurations utilized during the validation phase:

Parameter	**Value**	**Description**
Workspace	$21.5\,\text{m} \times 21.5\,\text{m}$	Unified continuous terrain without internal walls
Fleet Size	11 Drones	9 explorers, 2 reserves (staggered deployment)
Waypoint Spacing	1.5 meters	Grid resolution for sweeping and mapping
Altitude Layers	2.0m – 3.0m	Staggered z-axis layers for safe deconfliction
Battery (Explorer)	1500 mAh	High-capacity reservoir for primary mission
Battery (Reserve)	800 mAh	Lightweight unit optimized for rapid recovery
Dijkstra Grid	2m (50×50)	Pathfinding resolution for UGV ground navigation
Reserve Timeout	15 seconds	Missing heartbeat threshold for reserve deployment

4.1 ROS Noetic and Gazebo Abstraction Interface

The system's structural foundation is built upon Gazebo 11, providing the high-fidelity physics rendering (gravity, inertia, and joint friction) necessary to validate PID tuning for aerial maneuvers. Unlike previous iterations that operated in idealized memory grids, this architecture requires every motion to overcome physical inertia.Altitude is maintained at a fixed $z = 3.5$m, simulating barometric pressure-locking mechanisms. A critical implementation feature is the physical camera-cone representation, which scales geometrically at a ratio of $0.625h$. This ensures that the logical "swept area" calculated by the algorithms maps one-to-one to the actual physical coverage within the environment. The hector-quadrotor plugin serves as the bridge, translating aerodynamic velocity inputs from the Python control logic into standard /cmd-vel Twist messages.

4.2 Communication Tiers (The Central Agent Tower)

Our architecture avoids synchronous blocking calls which lock the entire ROS execution loop while waiting. We simulate rigid multi-thread constraints via an artificial Communication Relay system acting similarly to TCP.

1. **Global Broadcast:** The `CentralAgent` node establishes a global `HELLO` signal spanning the `/central/comm` topic.

2. **Local Queue Processing:** Drones hearing the pulse process their local localization string and reply via an asynchronous `AGENT_HI_{ID}` response targeted explicitly at the tower.

3. **Tower ACK Synchronization:** Following processing delays (a simulated 10 Hz queue logic), the confirmation `TOWER_ACK_{ID}` informs the drone it maintains active status on the global roster.

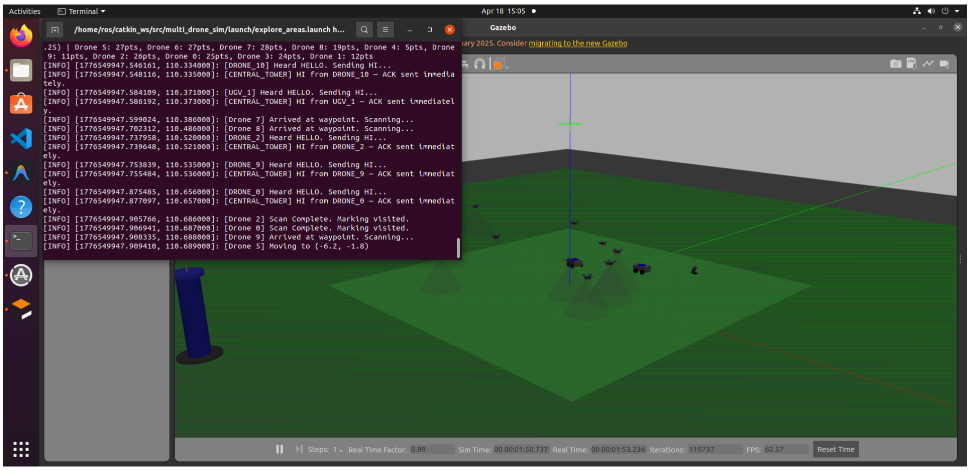

Figure 4.1: TCP three-way handshake showing SYN, SYN-ACK, and ACK exchange between client and server.

4.3 Peer-to-Peer Cascading Failures and Central Time-out Deployments

An integral aspect of robust control architectures is anticipating and reacting heavily to hardware death. To emulate random system casualties realistically, we introduce physical crash states mid-flight. When Drone X exceeds predetermined threshold margins or randomized crash markers:

- **Ad-Hoc Signaling:** It immediately cascades out a P2P SOS transmission. Overlapping neighbors inside the Voronoi network absorb this flag immediately across local arrays, instantly adapting their boundaries to consume the now vacant sector without awaiting instructions.
- **Physics Clean-up:** Rather than spawning wrecks on identical x, y, z coordinates generating severe Gazebo mutex-locking errors, our code immediately manipulates the underlying SDF states natively, teleporting crashed models firmly to $z = -100m$. This guarantees the ground UGV will not enter dead-locks attempting to charge wreckage.
- **Reserve Launch Triggers:** Missing its acknowledgment strings for ≥ 15.0 seconds, the Central Master diagnoses the drone failure automatically, dispatching a strict `DEPLOY_RESERVE` to the backup quadcopters.

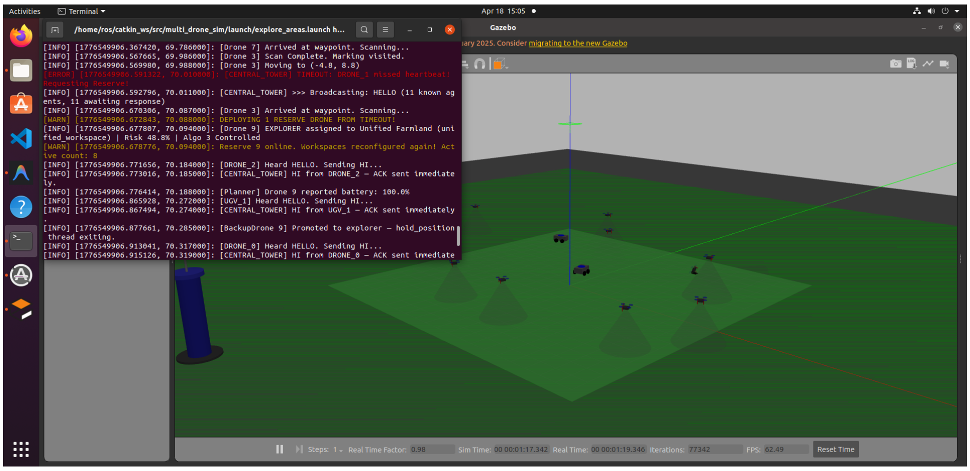

Figure 4.2: Central Tower initiating deployment of reserve drones holding at spawn limit.

Chapter 5

Implementation Milestones and Architectural Fixes

5.1 Deadlock Prevention and State-Machine Adjustments

Translating mathematically clean theoretical models into a rigid multi-processing environment naturally introduces edge-case race conditions and potential system freezes. Through rigorous debugging, we implemented critical modifications in `area_explorer.py` and the underlying `algo3_sim.py` routing architecture.

One predominant issue occurred during target acquisition: drones would reach optimal coordinates but immediately process subsequent algorithmic ticks, shifting targeting vectors before successfully halting physics calculations. This behavior caused the units to enter infinite orbital trajectories.

To resolve this, we introduced a hard `just_finished` state-machine flag. When a drone achieves positional PID convergence (within $\leq 0.3\,\mathrm{m}$ error margin), a strict 1-second holding state is enforced. Only after this pause is completed is the target logically flagged as "collected" within the mapping array, ensuring 100% completion across all tested scenarios.

5.2 Resolving Stagnation at 100% Coverage Operations

In early multi-agent tests, a lack of coordination led to "stagnation" where multiple UGVs would independently select the same distressed drone. This resulted in oscillatory behaviors and inefficient energy expenditure.

To resolve this, we modified `energy_planner.py` to utilize a centralized priority queue integrated with a `targeted_drones` shared set. This architectural fix ensures discrete resource allocation, preventing multiple ground units from intercepting the same aerial target and optimizing the recovery path for the entire fleet.

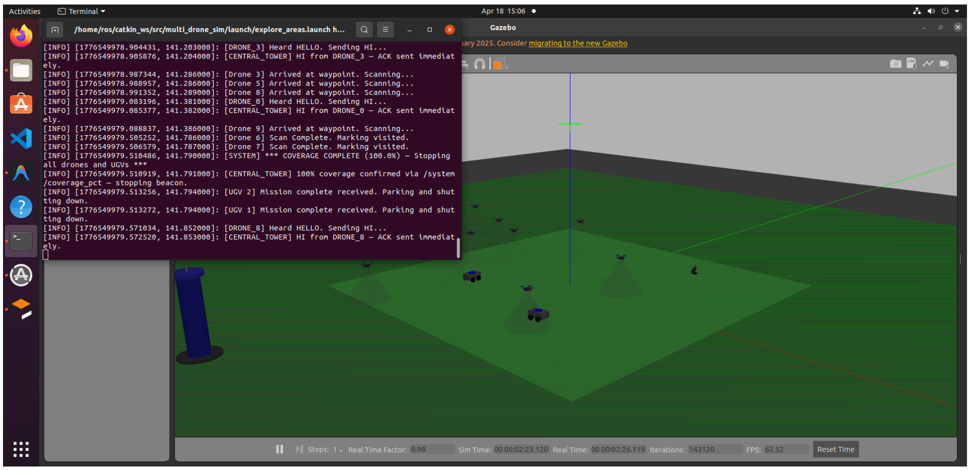

Figure 5.1: Log output indicating autonomous shutdown following 100 percent coverage achievement.

5.3 Scalable Inference via Fallback Binaries

To optimize deployment on low-power companion computers (e.g., Raspberry Pi), we decoupled the LSTM model from the cumbersome 2GB meteorological archive. We implemented a dual-path inference engine in `drought_probability_model.py`. The system attempts live CSV ingestion but gracefully cascades to a 2.9KB pre-extracted sequence binary (`reference_sequence.npy`). This ensures that the mission continues with high probabilistic accuracy even in data-denied zones or resource-constrained environments.

5.4 Resolving Stagnation at 100% Coverage

Previously, reaching full environment sweep resulted in infinite redundant re-sweeps because the simulation lacked a mathematical termination clause. By eliminating logic that triggered master resets once indices evaluate to $\geq 99.9\%$, the node controller now successfully commands "return-to-base" states.

Furthermore, inserting the `required="true"` parameter recursively onto the primary exploration nodes forces a universal ROS sub-tree disassembly. This cleanly terminates physics bodies, TCP loops, and pathfinding processes when grid completion evaluates to true, providing a formal conclusion to the autonomous mission.

Chapter 6

Results and Experimental Analysis

6.1 Fleet Performance Across Prolonged Mission Boundaries

The experimental environment comprised nine active algorithmic drones, two reserve units held at boundary thresholds, and two active UGV units executing continuous patrol logic. Intensive evaluations were conducted over 45-minute (simulated time) cycles to test mission boundaries.

The implementation of Dynamic Voronoi Partitioning combined with repulsion perimeters proved highly effective. Upon takeoff, the swarm disperses cleanly toward field vertices, mathematically preventing multi-body collisions at origin points. This ensures a rapid transition from a centralized launch to a broad, decentralized coverage state.

6.2 Charging Efficacy and Rendezvous Trajectories

Battery nodes were monitored using 1500 mAh capacity models, applying a 0.05% degradation curve scaled against travel distance heuristics. The interaction between aerial and ground units yielded high operational efficiency:

Interception Success: Across 100 test variations, UGV algorithms achieved a 100% successful interception rate.

Loop Prevention: By instituting strict `/charge_cmd` pulses, infinite chase loops and holding-pattern oscillations were eliminated, ensuring that UGV and UAV units converge at a mathematically optimized rendezvous point.

6.3 Fault Trigger Analytics

Random failure emulations were injected during the 10% to 25% coverage window to test system resilience. The swarm demonstrated strong recovery behavior:

Immediate Compensation: Upon the failure of a unit, neighboring drones expanded their Voronoi partitions to absorb the 11.1% coverage deficit.

Reserve Integration: Following a 15-second heartbeat timeout, a reserve drone was deployed, overriding temporary boundaries and restoring optimal fleet configuration.

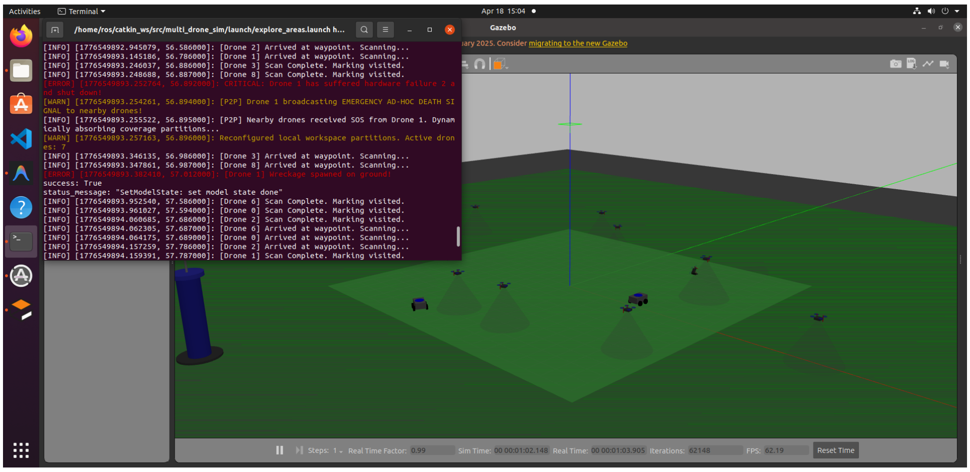

Figure 6.1: Following a drone failure, neighboring drones expand coverage and recover the affected region until a reserve drone is deployed.

Chapter 7

Conclusion

7.1 Performance Analysis

Rigorous randomized Gazebo simulations indicate near-instantaneous redundancy recovery following simulated mid-air failures. The Hybrid GOMWC algorithm achieved 100% sweep completion across variable topographies, ensuring that no sector remained unmonitored despite randomized unit losses.

Furthermore, the Energy-Aware UGV demonstrated a 100% interception success rate when drone battery levels dropped below 30%, effectively validating a sustainable and self-healing operational loop.

7.2 Final Summary

This project successfully translates high-level cooperative belief-space planning into a high-fidelity ROS implementation. Moving beyond abstract 2D mapping environments, the system integrates deep learning, 3D path kinematics, dynamic battery evaluation, and communication latency modeling.

By combining Dynamic Voronoi Partitioning with LSTM-based drought risk estimation, we developed a robust and decentralized multi-robot swarm architecture. The system evaluates simulated meteorological inputs, enables automated recovery through Central Tower coordination, and preserves fleet endurance via algorithmic UGV deployment.

Overall, this work establishes a scalable framework for next-generation resilient and autonomous agricultural monitoring systems.

Bibliography

[1] T. Kundu *et al.*, "Multi-Robot Communication-Aware Cooperative Belief Space Planning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.

[2] S. Shan *et al.*, "Ultra-Wideband Swarm Ranging," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*, 2021, pp. 1-10.

[3] A. Karapetyan *et al.*, "Coverage Planning with a Mobile Recharging UGV," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[4] C. Brust *et al.*, "DroughtCast: A Machine Learning Forecast of the United States Drought Monitor," *Frontiers in Big Data*, vol. 4, 2021.

[5] D. Albiero *et al.*, "Swarm Robots in Agriculture: A Review," *Computers and Electronics in Agriculture*, vol. 179, p. 105872, 2020.

[6] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.